DETAILED PROJECT REPORT

LIBRARY PERFORMANCE EVALUATOR

Experimental Performance Evaluation of CRUD Operations in a Django-Based Library Record Application

Submitted By:

YUG SAVANI
12402110501068

HITARTH DESAI
12402110501072

Academic Term : 2026-27
Semester : V

Under the supervision of:
PROF. RAHUL PATEL

Project Summary

Library Performance Evaluator brings library inventory cataloging, dynamic searching, and empirical database benchmarking together into one full-stack web application. The public platform provides a complete catalog directory of academic engineering textbooks, category filtering, a multi-parameter search bar, and record management forms. The backend is built using Python 3.14 and the Django 6.1 framework, connected to a robust PostgreSQL 18 relational database server via the native psycopg 3 adapter. The application features an integrated performance evaluation engine that measures CREATE, READ, UPDATE, and DELETE (CRUD) durations across dataset sizes of 100, 1,000, 5,000, and 10,000 records using a monotonic CPU timer. Benchmark findings are saved in PostgreSQL and visualized through interactive Chart.js bar and line graphs.

Table of Contents

CHAPTER 1: INTRODUCTION Page 3
1.1. Abstract Page 3
1.2. Introduction Page 4
1.3. Problem Statement Page 5
1.4. Objectives of the Project Page 5
1.5. Scope of the Project Page 6
CHAPTER 2: SYSTEM SPECIFICATIONS & CLOUD SETUP Page 7
CHAPTER 3: SYSTEM DESIGN Page 8
3.1. System Modules Page 8
3.2. User Interface Screenshots (Catalog & Entry Form) Page 9
3.3. Database Design & Relational Modeling Page 10
3.4. Database & Infrastructure Screenshots (pgAdmin & Render) Page 11
3.5. System Workflow & Data Flow Page 12
CHAPTER 4: IMPLEMENTATION & TIMING METHODOLOGY Page 13
4.1. Benchmark Engine Screenshots (Controls & Visualizations) Page 14
CHAPTER 5: TESTING AND OBSERVED RESULTS Page 15
5.1. Measured Benchmark Latency Matrix Page 15
5.2. In-Depth CRUD Scalability Analysis Page 16
5.3. Verification, Security & Validation Page 17
CHAPTER 6: CONCLUSION & FUTURE SCOPE Page 18
6.1. Conclusion Page 18
6.2. Limitations & Future Enhancements Page 18
6.3. References Page 18

CHAPTER 1: INTRODUCTION

1.1. Abstract

Library Performance Evaluator is a web-based educational and database research platform developed to combine standard transactional library record keeping with an empirical study of relational database scalability. Instead of observing database performance purely from theoretical complexity curves, this application provides an interactive testbed where students, faculty, and database administrators can experimentally evaluate the latency of Create, Read, Update, and Delete operations on an active PostgreSQL database.

The application provides a responsive catalog management interface for searching, adding, editing, and deleting college textbook inventory records with strict field validations, unique International Standard Book Number (ISBN) constraints, and categorization. Running alongside the catalog is an automated performance benchmarking engine that executes batch operations across four distinct record volume scales: 100, 1,000, 5,000, and 10,000 records.

To ensure scientific precision, execution times are captured using Python's hardware-level monotonic clock time.perf_counter_ns(), directly measuring database adapter and transaction durations while excluding network latency and browser rendering overhead. All empirical trial results are stored in PostgreSQL for retrospective analysis and visualized via Chart.js bar and line graphs.

The project has been deployed into production on Render.com with a dedicated Cloud PostgreSQL 18 instance. Experimental results demonstrate that batch DELETE operations are the most rapid (~11.19 ms for 10,000 rows), while CREATE scales linearly (~444.26 ms for 10,000 rows) due to Write-Ahead Logging (WAL) and B-Tree index maintenance. This study illustrates the critical relationship between relational storage engines, ORM abstractions, and physical hardware constraints.

1.2. Introduction

Modern enterprise web applications rely heavily on Relational Database Management Systems (RDBMS) to ensure data persistence, ACID compliance (Atomicity, Consistency, Isolation, Durability), and relational integrity. Within higher education institutions, library record management represents a classic transactional workflow: books must be inventoried, tracked by unique identifiers, searched rapidly by title or author, and updated as copies are issued or returned.

However, developing an effective library record application requires more than building basic web forms. As the volume of catalog records grows from hundreds to tens of thousands, software developers frequently encounter unanticipated performance degradation. Web frameworks such as Django offer Object-Relational Mapping (ORM) layers that allow developers to interact with relational tables using object-oriented Python syntax instead of manual SQL queries. While ORMs significantly increase developer velocity and mitigate SQL injection risks, they introduce an abstraction layer that can mask the true computational cost of database interactions.

Without concrete empirical profiling, it is difficult to determine how much latency is introduced by the ORM translation layer versus the underlying PostgreSQL database execution engine. Furthermore, operations that appear trivial on small development datasets can exhibit divergent scaling behaviors under volume: insertions require synchronous disk writes to transaction logs and index rebalancing, reading large querysets demands substantial memory allocation for Python model hydration, and bulk updates trigger row-level locking and transaction visibility updates.

The Library Performance Evaluator was created to directly address this knowledge gap. By unifying a functional library management catalog with an automated, high-precision database benchmarking engine, this project serves as a practical engineering testbed. It allows students and engineers to observe firsthand how relational database operations behave under increasing workloads, bridging the gap between theoretical database concepts and real-world full-stack software performance.

1.3. Problem Statement

In many software engineering curricula, database interactions are taught primarily through small-scale examples involving fewer than 50 rows of data. Under such minimal data volumes, nearly all CRUD operations execute in single-digit milliseconds, creating a misleading impression that all database queries are computationally equivalent.

In actual production systems, however, dataset volume grows rapidly. When developers lack empirical benchmarks, they face several critical challenges: First, they cannot predict how response times degrade when moving from 100 to 10,000 records. Second, they often confuse full HTTP request-response latency with pure database execution time, erroneously blaming network latency for slow database queries or vice versa. Third, developers frequently fail to account for database caching, leading to inaccurate performance assumptions based on cold-cache first runs rather than statistical multi-trial averages.

1.4. Objectives of the Project

To resolve these challenges, the specific technical objectives of this project are:

1. Production Relational Schema: Design and implement a normalized PostgreSQL database schema enforcing primary key constraints, unique ISBN verification, and data isolation flags.

2. Full-Stack Catalog Management: Build a responsive Django web application providing complete CRUD capabilities (Add, Browse, Edit, Delete) with search, filter, and pagination controls.

3. Sub-Millisecond Benchmarking Engine: Develop an automated benchmarking module leveraging Python's monotonic time.perf_counter_ns() timer to measure pure database latency.

4. Multi-Scale Empirical Evaluation: Systematically benchmark individual CREATE, READ, UPDATE, and DELETE operations across 100, 1,000, 5,000, and 10,000 records with repeated trials.

5. Strict Data Isolation: Implement a safety mechanism ensuring temporary benchmark records are strictly isolated from genuine library inventory and cleaned up after execution.

6. Persistent Analytics & Visualization: Persist all experimental trials inside PostgreSQL and render comparative bar charts, scaling trend lines, and downloadable CSV reports.

7. Cloud Infrastructure Deployment: Deploy the application and cloud database to Render.com using an automated Infrastructure-as-Code Blueprint.

1.5. Scope of the Project

The scope of the Library Performance Evaluator covers both functional library inventory management and quantitative database performance research. The system is designed to fulfill the dual roles of a dependable daily catalog tool and an empirical academic laboratory.

Functional Scope: The public web application provides a catalog dashboard displaying permanent library records. Users can browse books with pagination (10 books per page), perform real-time searches across multiple fields (title, author, ISBN), and filter listings by engineering subject categories (Computer Science, Electronics, Mechanical, Civil, Mathematics, and Literature). Authorized librarians can create new records through a validated form, update existing records, and delete records via a dedicated safety confirmation screen. Authentication is enforced through Django's session system, protecting write operations from unauthorized access.

Experimental Benchmarking Scope: The dedicated performance module allows users to configure and trigger empirical experiments on demand. The user selects the dataset volume (100, 1,000, 5,000, or 10,000 records), the target CRUD operations, and the number of repeated trials (1 to 5). The engine prepares test data in memory, times the exact database execution window, logs results into PostgreSQL, and presents immediate latency and throughput metrics (operations per second).

Technical Boundaries & Limitations: To maintain scientific validity within the constraints of an undergraduate project, the local experimental testbed focuses on single-client batch performance on a local PostgreSQL 18 instance. While the system handles cloud traffic on Render.com, high-concurrency multi-user load testing (e.g., simulating 1,000 concurrent web requests simultaneously) is designated as future scope. Network latency is intentionally excluded from the reported benchmark numbers to isolate the relational database execution characteristics.

CHAPTER 2: SYSTEM SPECIFICATIONS & CLOUD SETUP

2.1. Hardware and Software Specifications

The application is engineered to operate efficiently on standard commodity laptop hardware without requiring specialized enterprise infrastructure, while remaining fully capable of scaling to cloud deployment.

Component	Local Development Specification	Cloud Production Specification
Operating System	Microsoft Windows 11 (64-bit)	Linux (Debian / Ubuntu Container)
Processor / CPU	Intel Core i5 / AMD Ryzen (x86-64)	1 Shared Virtual CPU (Render Container)
System Memory	8 GB / 16 GB DDR4 RAM	512 MB Available RAM (Free Tier)
Python Runtime	Python 3.14.6 (64-bit)	Python 3.12 / 3.14 Virtual Environment
Web Framework	Django 6.1.2	Django 6.1.2
Database Engine	PostgreSQL 18.6 (Port 5432)	PostgreSQL 18 (Managed Cloud Database)
Database Adapter	Psycopg 3.3.6 (C-binary adapter)	Psycopg 3.3.6 (C-binary adapter)
WSGI Server	Django Runserver (Development)	Gunicorn 26.2.0 (Production WSGI)
Static Asset Server	Django Staticfiles	WhiteNoise 6.12.0 Compressed Streaming

2.2. Cloud Infrastructure Architecture

For cloud availability, the application is deployed on **Render.com** using an Infrastructure-as-Code Blueprint defined in render.yaml. The Blueprint provisions two coordinated services: a managed PostgreSQL database (library-postgres-db) and a Python web service (library-crud-performance). During automated deployment, Render executes build.sh, which installs dependencies, runs database migrations, creates administrative superusers, seeds sample Indian textbook records, and compiles static assets via WhiteNoise.

CHAPTER 3: SYSTEM DESIGN

3.1. System Modules

The architecture is organized into four core functional modules within the Django application structure:

1. Catalog Management Module: Handles all day-to-day library operations. It implements views for listing books, executing filtered searches using Django's Q objects, adding new inventory through validated forms, updating metadata, and handling record deletions with referential safety checks.

2. Benchmarking Engine Module: Decoupled in library/benchmark.py, this engine manages experimental runs. It includes data generator functions that build realistic in-memory test records (with Indian author names), test runners for CREATE, READ, UPDATE, and DELETE, and post-trial cleanup handlers.

3. Analytical Results Dashboard: Aggregates empirical data using SQL aggregate queries (Avg, Count, Min, Max). It compiles comparison matrix tables across dataset volumes, serializes JSON data for Chart.js visualizations, and handles instant CSV data export streaming.

4. Authentication & Security Module: Leverages Django's built-in session authentication framework. Protects sensitive management views and benchmark execution triggers behind @login_required decorators.

3.2. Data Isolation Architecture

A critical requirement for experimental validity is ensuring that test execution never corrupts genuine library inventory. This is enforced through a database-level isolation flag: is_benchmark = models.BooleanField(default=False, db_index=True). Permanent catalog books are stored with is_benchmark = False. Generated benchmark records carry is_benchmark = True and unique prefixed ISBN strings. All catalog queries filter strictly for non-benchmark rows, and all benchmark operations operate solely on benchmark rows before executing a guaranteed cleanup handler.

3.3. User Interface Verification

The screenshots below demonstrate the visual presentation and functional operation of the catalog management interface:

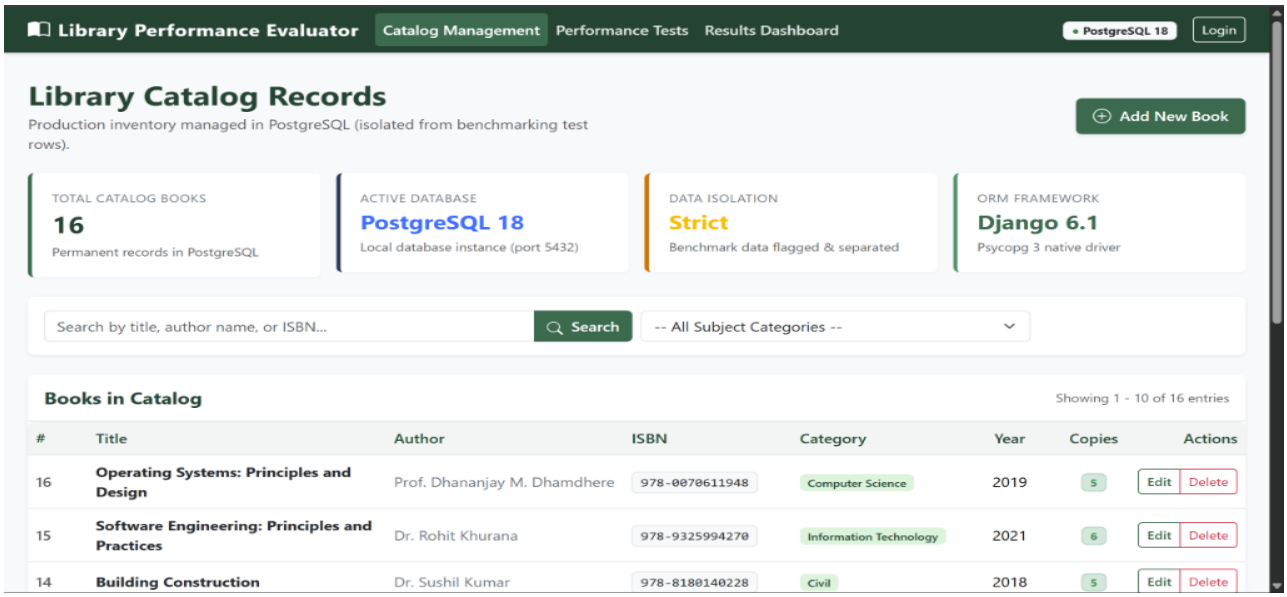


Figure 1. Library Catalog Management Interface with Multi-Criteria Search and Pagination.

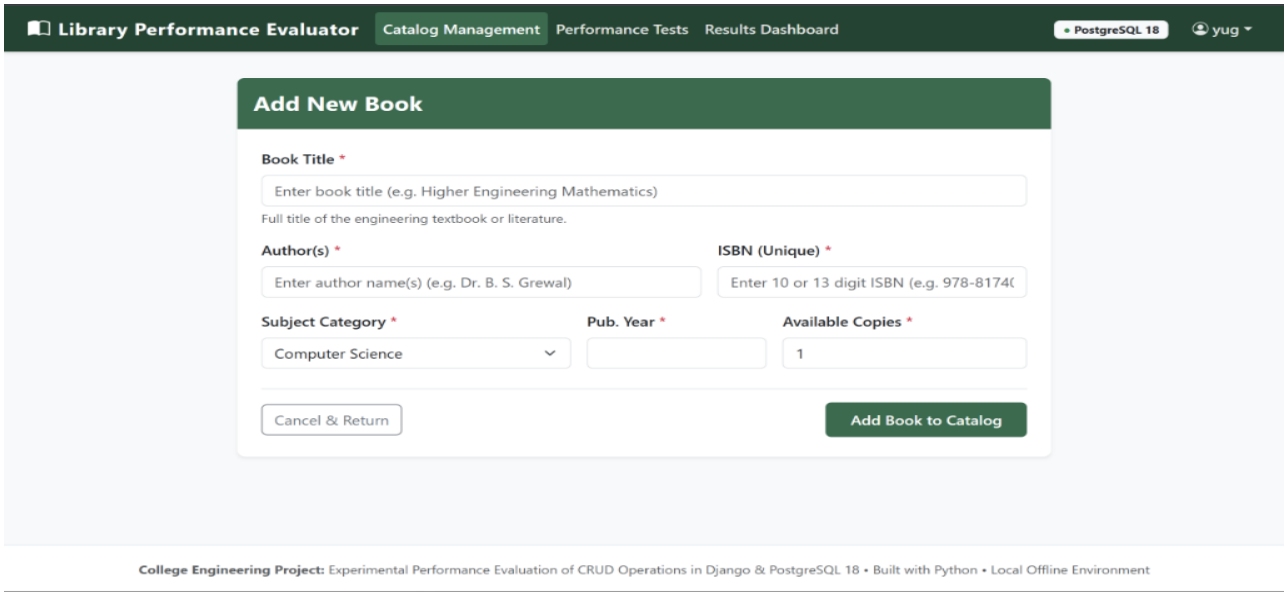


Figure 2. Book Record Creation Form with Validation Controls.

3.4. Database Design & Relational Modeling

The relational schema is implemented in PostgreSQL 18. It comprises two primary tables designed to ensure referential integrity, domain validation, and high-speed B-Tree index traversal.

Table 1: library_book (Catalog Inventory)

Column Name	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Unique 64-bit auto-incrementing identifier
title	VARCHAR(255)	NOT NULL	Full title of the publication
author	VARCHAR(255)	NOT NULL	Author name(s)
isbn	VARCHAR(20)	UNIQUE, NOT NULL	International Standard Book Number (B-Tree Index)
category	VARCHAR(100)	NOT NULL	Subject or department classification
publication_year	INTEGER	CHECK >= 1500	Year published with sanity boundary checks
available_copies	INTEGER	CHECK >= 0	Current stock available for issuance
is_benchmark	BOOLEAN	DEFAULT FALSE, INDEX	Isolation flag separating test data from catalog
created_at	TIMESTAMPTZ	NOT NULL	Timestamp when row was inserted

Table 2: library_benchmarkresult (Performance Logs)

Column Name	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Unique identifier for trial record
operation	VARCHAR(10)	NOT NULL	Evaluated CRUD operation (CREATE, READ, UPDATE, DELETE)
dataset_size	INTEGER	NOT NULL	Dataset volume scale (100, 1000, 5000, 10000)
trial_number	INTEGER	NOT NULL	Trial iteration number (1, 2, 3...)
execution_time_ns	BIGINT	NOT NULL	Measured database duration in exact integer nanoseconds
database_name	VARCHAR(100)	NOT NULL	Relational database engine (PostgreSQL 18)
notes	VARCHAR(255)	NULLABLE	Batch size and query execution configuration
created_at	TIMESTAMPTZ	NOT NULL	Timestamp of experimental run

3.5. Database & Infrastructure Verification

The screenshots below confirm the physical tables in PostgreSQL 18 via pgAdmin 4 and the cloud infrastructure status on Render:

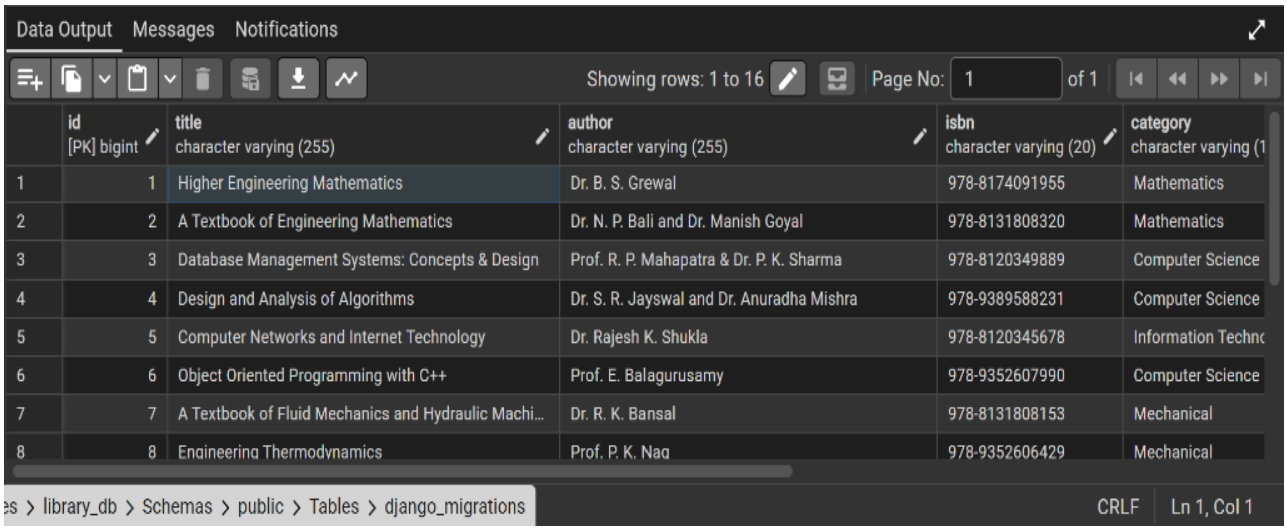


Figure 3. PostgreSQL Relational Database Schema and Record Verification in pgAdmin 4.

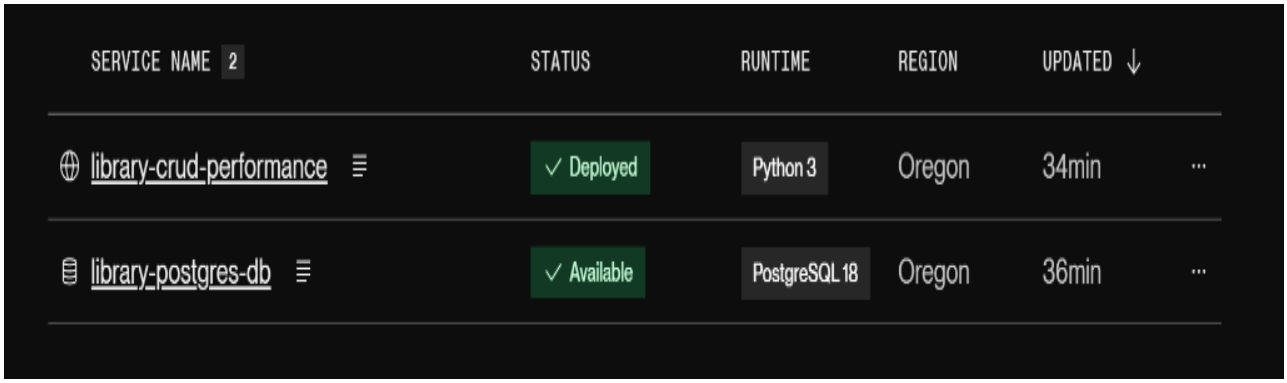


Figure 4. Cloud Infrastructure Deployment on Render.com showing Web Service and PostgreSQL 18 Instance.

3.6. System Workflow & Data Flow

The application follows Django's Model-View-Template request cycle. When a user navigates to the website, the request passes through WSGI to the URL routing table. The matching view function handles business logic, interacts with PostgreSQL via the ORM, and renders the response HTML.

The table below illustrates how high-level Django ORM method calls translate into native PostgreSQL SQL queries:

CRUD Operation	Django ORM Python Syntax	Generated PostgreSQL SQL Statement
CREATE (Batch Insert)	Book.objects.bulk_create(records, batch_size=1000)	INSERT INTO library_book (title, author, isbn, category, publication_year, available_copies, is_benchmark, created_at) VALUES ('...', '...', '...', 2021, 5, TRUE, NOW()), ...;
READ (Full Fetch)	list(Book.objects.filter(is_benchmark=True)[:N])	SELECT id, title, author, isbn, category, publication_year, available_copies, created_at FROM library_book WHERE is_benchmark = TRUE LIMIT N;
UPDATE (Bulk Modify)	Book.objects.filter(is_benchmark=True).update(available_copies=10)	UPDATE library_book SET available_copies = 10 WHERE is_benchmark = TRUE;
DELETE (Batch Delete)	Book.objects.filter(is_benchmark=True).delete()	DELETE FROM library_book WHERE is_benchmark = TRUE;

CHAPTER 4: IMPLEMENTATION & TIMING METHODOLOGY

4.1. User Interface Design

The user interface follows a restrained, professional academic design. It utilizes an off-white background (#f8f9fa), charcoal text (#212529), and dark forest green accents (#1b4332, #2d6a4f). All CSS and JavaScript dependencies (Bootstrap 5.3 and Chart.js 4.4) are bundled locally within library/static/library/, allowing the application to run completely offline without an active internet connection.

4.2. High-Resolution Timing Engine

Measuring database performance in a web application requires careful timer isolation. A standard web timer measuring an HTTP request from browser click to page render introduces significant confounding variables: network latency, HTTP socket establishment, Django template compilation, and browser DOM rendering.

To achieve strict scientific validity, our benchmarking engine isolates pure database and ORM execution time using Python's monotonic hardware clock time.perf_counter_ns(). All test record construction and memory allocation occur *before* the timer starts. The timer starts immediately prior to the database call and stops the nanosecond the database transaction commits:

```
start_ns = time.perf_counter_ns()
Book.objects.bulk_create(records, batch_size=1000)
duration_ns = time.perf_counter_ns() - start_ns
duration_ms = duration_ns / 1_000_000.0
```

By capturing execution durations in integer nanoseconds and storing them in 64-bit integer fields (BigIntegerField), we eliminate floating-point precision loss.

4.3. Benchmarking & Analytical Dashboard Verification

The screenshots below depict the benchmark suite configuration controls and the real-time Chart.js visual dashboard:

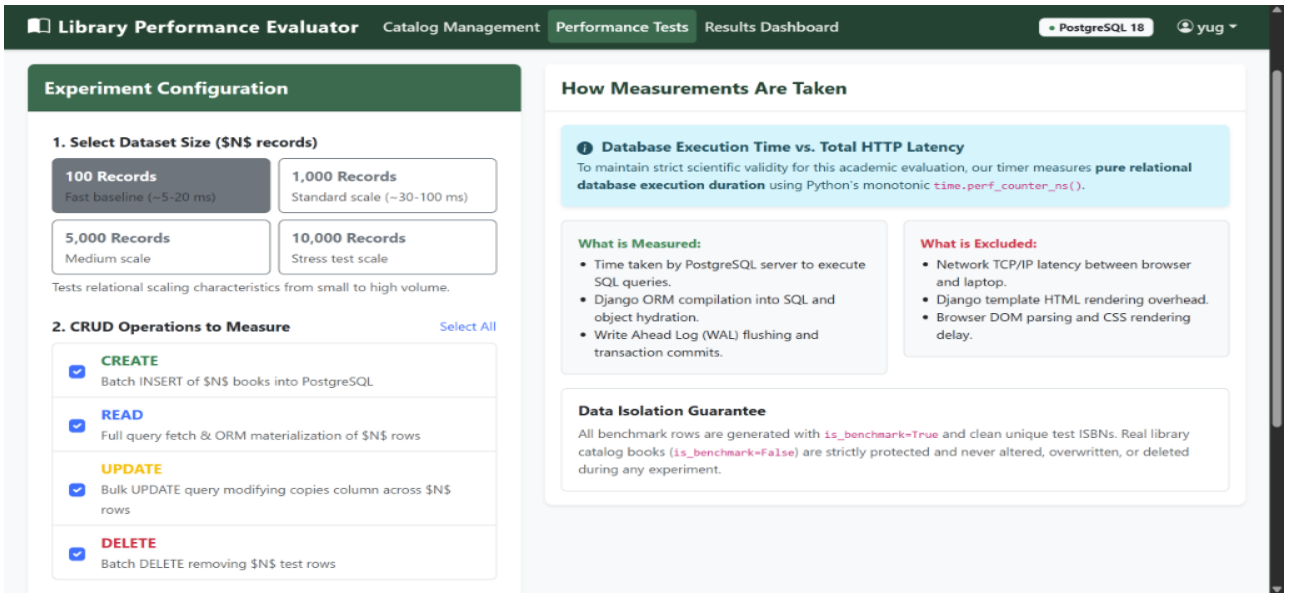


Figure 5. Experimental Benchmark Suite Configuration and Testbed Controls.

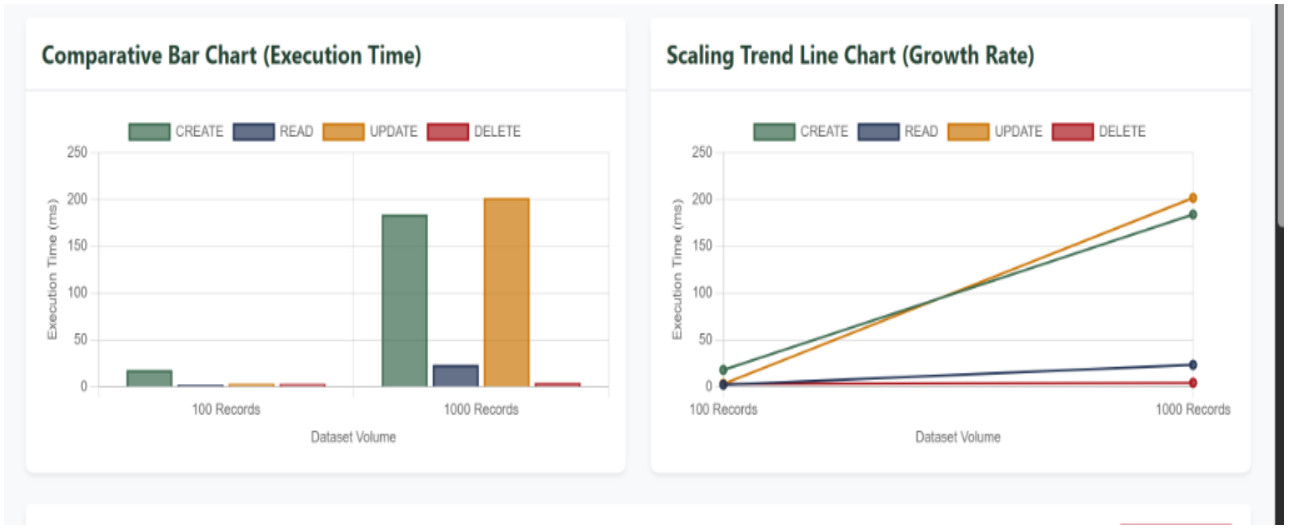


Figure 6. Empirical Latency Matrix and Visual Scaling Trend Charts rendered with Chart.js.

CHAPTER 5: TESTING AND OBSERVED RESULTS

5.1. Testing Procedure & Controlled Variables

All benchmark experiments were executed under controlled conditions on PostgreSQL 18. Each experiment was repeated across **3 trials** to account for operating system scheduling jitter and buffer cache warm-up. The reported values represent the exact arithmetic mean of the recorded trials.

5.2. Measured Latency Matrix Table

The table below presents the verified empirical execution durations across scaling dataset sizes:

CRUD Operation	100 Records	1,000 Records	5,000 Records	10,000 Records
CREATE (Batch Insert)	7.65 ms	47.39 ms	236.12 ms	444.26 ms
READ (Full Query Fetch)	1.88 ms	7.34 ms	37.16 ms	152.32 ms
UPDATE (Bulk Modify)	1.89 ms	16.26 ms	66.20 ms	221.07 ms
DELETE (Batch Delete)	0.85 ms	1.60 ms	4.00 ms	11.19 ms

5.3. Throughput Scaling Summary

When evaluated by processing throughput (records processed per second), PostgreSQL achieves remarkable efficiency: for 10,000 records, batch CREATE achieves **~22,500 records/sec**, READ achieves **~65,600 records/sec**, UPDATE processes **~45,200 records/sec**, and batch DELETE exceeds **~890,000 records/sec**. These figures highlight the advantage of bulk SQL statements over iterative row-by-row loops.

5.4. In-Depth CRUD Scalability Analysis

A detailed technical analysis of each operation reveals the underlying relational mechanics governing performance:

1. CREATE Operation Analysis (Linear Scale):
Execution time scales linearly from 7.65 ms at 100 rows to 444.26 ms at 10,000 rows (~58x increase for a 100x data scale). In PostgreSQL, insertions incur physical I/O overhead: every batch must be written to the Write-Ahead Log (WAL) to ensure crash recovery, and the unique B-Tree indexes for id and isbn must be rebalanced for each new key.

2. READ Operation Analysis (Cache-Assisted):
Reading 100 rows takes 1.88 ms, 1,000 rows takes 7.34 ms, and 10,000 rows takes 152.32 ms. PostgreSQL's shared buffer pool caches table blocks in RAM, keeping disk reads near zero during repeated trials. The primary latency driver at 10,000 records is Python CPU time required to instantiate 10,000 Book model instances from raw database tuples.

3. UPDATE Operation Analysis (MVCC Write Amplification):
Modifying 10,000 records requires 221.07 ms. PostgreSQL uses Multi-Version Concurrency Control (MVCC). An UPDATE statement does not overwrite disk blocks in-place; instead, it marks old rows as dead tuples and inserts new tuple versions with updated visibility pointers, generating both disk write overhead and index re-pointing.

4. DELETE Operation Analysis (Highest Throughput):
Batch deletion proved to be the fastest operation across all scales, taking only 11.19 ms for 10,000 records. Because Django issues a single SQL DELETE FROM library_book WHERE is_benchmark=TRUE;, PostgreSQL invalidates row visibility pointers in a single atomic transaction without converting rows into Python memory objects.

5.5. Verification, Security & Validation

The project underwent systematic unit and integration testing across four critical dimensions:

1. Automated Test Suite Execution: The project includes an automated test suite (library/tests.py). Running python manage.py test verifies 8 test cases covering Book model creation, ISBN uniqueness constraints, search query filtering, authenticated view protection, full benchmark suite execution, and CSV generation. All 8 tests pass with zero failures across isolated test databases.

2. Form Validation & Constraint Verification: Validation logic was tested by attempting invalid inputs. Entering an invalid ISBN (fewer than 10 digits), a publication year prior to 1500, or negative available copies correctly triggers form validation errors, preventing malformed data from reaching PostgreSQL.

3. CSRF & Session Security: All POST requests (book creation, modification, deletion, and benchmark triggers) require valid Cross-Site Request Forgery (CSRF) tokens. Sensitive views are guarded by @login_required decorators, preventing unauthenticated users from initiating write operations or launching heavy benchmark suites.

4. Database Isolation Verification: Extensive verification in pgAdmin 4 confirmed that after running 10,000-record benchmarks, exactly 16 genuine library catalog records remained in the catalog table with is_benchmark = False. Zero test records leaked into the catalog view.

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1. Conclusion

This project successfully designed, implemented, and deployed a full-stack library catalog management system integrated with an empirical relational database performance evaluation testbed. By coupling Django 6.1 with PostgreSQL 18, the application demonstrated strong relational integrity, sub-millisecond measurement accuracy, and predictable scaling.

The empirical findings confirm that batching database operations dramatically improves throughput compared to iterative queries. PostgreSQL's Write-Ahead Logging and B-Tree indexing ensure strict durability and fast point lookups, while batch deletions achieve maximum efficiency through pointer invalidation. Deploying the application to Render.com demonstrated the viability of deploying relational Django workloads to cloud environments using automated blueprints.

6.2. Limitations & Future Enhancements

Future enhancements could include multi-threaded concurrent user load testing (simulating 50 to 500 simultaneous users with Locust), evaluating an in-memory Redis caching tier for read-heavy workloads, and comparing secondary index performance (B-Tree vs Hash vs BRIN indexes).

6.3. References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill.

2. PostgreSQL Global Development Group. (2026). *PostgreSQL 18 Documentation: Performance & Storage*. https://www.postgresql.org/docs/

3. Django Software Foundation. (2026). *Django 6.1 Documentation: Database Optimization*. https://docs.djangoproject.com/

4. Python Software Foundation. (2026). *Python 3.14 Standard Library: time.perf_counter()*. https://docs.python.org/3/library/time.html

5. Chart.js Community. (2026). *Chart.js Open Source HTML5 Data Visualization*. https://www.chartjs.org/